

<Weight & Bias>

Weight (W)

→ 압축데이터가 출력에 얼마나 영향을 미치는가?

- 기울기 조절: $y = WX$ 에서

W 가 크면 입력변화에 대해 출력이 민감

W 가 작으면 0에 가까우면 입력이 아무리 변해도 출력변화 미미

- 특징(중요도 제어): 정답을 맞추는데 중요한 입력에 큰 W 부여
그게 아닌 노이즈 성분에 0에 가까운 W

Bias (B)

→ 기본적으로 잘고하는 조건, 즉 W 를 아무리 곱해도 해결할 수 없는 '위치적 한계' 보완

- 특징: ANN은 뉴런과 유사 특징 임계값을 넘어야 한다.

그걸 조절하고, nn.Linear 두에는 보통 nn.ReLU가 붙어있음

ReLU는 입력값이 0보다 크면 통과시키고, 0 이하는 0으로

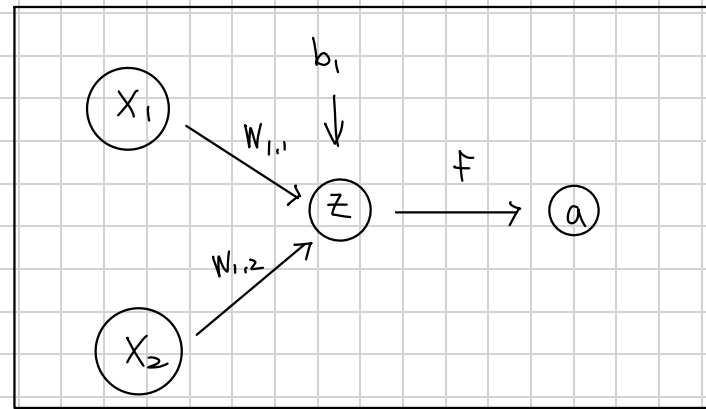
여기서 Bias가 정답값에 중요한 것은 통과시켜주도록 해줌

(If) 편향이 없다면?

→ 오로지 X 와 W 로만 활성화 여부 결정한다.

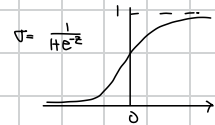
여기서 WX 값이 모두 음수라면, 모두 활성화되지 않는

신경망 먹통 현상 발생



f: 활성화함수

- Sigmoid



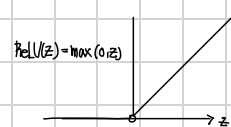
특징: 출력이 무조건 0과 1 사이

단점: 이 뉴런이 활성화될 확률 파악

현재: 출력층에서 이전 뉴런 (0/1) 확률 파악

단점: z 값이 커지거나 작아지면 기울기 근방 0이 됨.
역전파 과정 힘들.

- ReLU

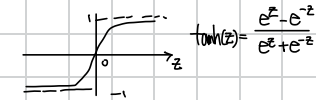


특징: 단순 비교 계산 비용,

양수영역에서 미분값 항상 1 → 아무리 큰 값으로 오차 줄어들지 않고 원본 그대로 역전파 가능

단점: 음수 표현 불가

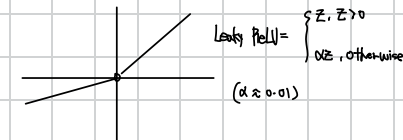
- Hyperbolic Tangent



특징: 원점 대칭, 출력값 범위 -1 ~ 1

단점: 기울기 소실 문제

- Leaky ReLU



특징: ReLU 사용 후 모델 성능이 더 이상 안올때 사용

음수영역에 아주 작은 기울기 (0.01) 곱해서 출력

→ 음수 영역에서도 오차를 계속 전달하며 학습

[3단계 코드 분석]

수식적인 과정

self.net work = sequential(~)



< 입력층 >

입력값 x가 nn. Linear(1, 64)를 만나면
행렬 곱셈이 일어남.

→ 통과시 하나의 입력값으로
서로 다른 가중치와 편이를 가진 64개의 직선 세트

$$z^{(1)} = \begin{bmatrix} z_1 \\ \vdots \\ z_{64} \end{bmatrix} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_{64} \end{bmatrix} \times [x] + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_{64} \end{bmatrix} = \begin{bmatrix} w_1 x + b_1 \\ w_2 x + b_2 \\ \vdots \\ w_{64} x + b_{64} \end{bmatrix}$$

→ 이제 64개 직선 각각의 ReLU 함수 $\max(0, z)$ 적용

$$a^{(1)} = \text{ReLU}(z^{(1)}) = \begin{bmatrix} \max(0, w_1 x + b_1) \\ \max(0, w_2 x + b_2) \\ \vdots \\ \max(0, w_{64} x + b_{64}) \end{bmatrix}$$

< 첫번째 은닉층 >

→ 이번에는 $a^{(1)}$ 을 nn. Linear(64, 64) 대입

$$z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ \vdots \\ z_{64}^{(2)} \end{bmatrix} = \begin{bmatrix} w_{1,1}^{(2)} & \dots & w_{1,64}^{(2)} \\ w_{2,1}^{(2)} & \dots & w_{2,64}^{(2)} \\ \vdots & \ddots & \vdots \\ w_{64,1}^{(2)} & \dots & w_{64,64}^{(2)} \end{bmatrix} \begin{bmatrix} a_1^{(1)} \\ \vdots \\ a_{64}^{(1)} \end{bmatrix} + \begin{bmatrix} b_1^{(2)} \\ b_2^{(2)} \\ \vdots \\ b_{64}^{(2)} \end{bmatrix}$$

→ 이걸 다시 ReLU 대입

$$a^{(2)} = \text{ReLU}(z^{(2)}) = \text{ReLU} \left(\begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ \vdots \\ z_{64}^{(2)} \end{bmatrix} \right) = \begin{bmatrix} \max(0, z_1^{(2)}) \\ \max(0, z_2^{(2)}) \\ \vdots \\ \max(0, z_{64}^{(2)}) \end{bmatrix}$$

↓ 동일하게 [두번째 은닉층
세번째 은닉층] 진행

< 출력층 >

→ 여기에서는 nn. Linear(64, 1)를 통과

즉, 64개를 입력 받아 1개의 값을 출력

$$[\hat{y}] = [w_1^{(5)} \ w_2^{(5)} \ \dots \ w_{64}^{(5)}] \begin{bmatrix} a_1^{(4)} \\ a_2^{(4)} \\ \vdots \\ a_{64}^{(4)} \end{bmatrix} + b^{(5)}$$

$$\Rightarrow \therefore \hat{y} = (w_1^{(5)} a_1^{(4)} + w_2^{(5)} a_2^{(4)} + \dots + w_{64}^{(5)} a_{64}^{(4)}) + b^{(5)}$$

[카이밍 최적화]

< 카이밍 초기화 가중치 대입 과정 > ①

① 입력 노드 수 (n_{in}) 확인

→ $n_{in} = 1$ (x값 1개 받아서, 64개로 확장)

② 표준편차 계산

$$\sigma = \sqrt{\frac{2}{n_{in}}} = \sqrt{\frac{2}{1}} = \sqrt{2} \approx 1.414$$

즉, 평균 0, 표준편차 1.414 인 가우시안 분포로 임의 개입

③ 실제 가중치 대입

정규분포 특성상 표준편차 범위인 ± 1.414 안쪽 숫자들이

분할 확률이 68%로 가장 높기 때문에, 이를 바탕으로

$W_1 \sim W_{64}$ 까지 채워짐

④ 편향 초기화

→ 가중치 이후에 편향(b)는 모든 방에

0으로 채워 초기화

Why? → 시작부터 무작위 값을 주면

처음부터 예측선이 이상한 곳으로 가기 때문에,

안전하게 0에서 시작하여 학습을 통해 수정

수식: $Var(W) = \frac{2}{n_{in}}$ (n_{in} 은 입력 노드의 개수)

→ 가중치 평균이 0
→ 출력데이터 평균이 0 근처를 유지해
→ 신호 왜곡없이 깊은 층까지 데이터 전달
→ 발생원인

표준편차: $\sigma = \sqrt{\frac{2}{n_{in}}}$

즉, 출력데이터가 평균이 0 근처에 유지하지 못한다면,

① 음수 풀림

ReLU에서 다 0으로

② 양수 풀림

매층을 거쳐 가중치와 곱하여

숫자가 점점 커지게 되고,

메모리가 감당불가

③ 층이 깊어지면

전반은 죽고
전반은 살고

→ 전체적인 스케일 유지

해서 데이터 온전히 보존 실패

[카이밍 초기화]

< 레이어를 통과한 매의 분산 변화 수식 증명 > ②

출력신호 $Y = WX$ 의 분산을 가정

(편의상 편향 $b=0$, W 와 X 는 독립이며 평균이 0이라 가정)

초기 0으로 세팅

① 서로 독립인 분산, $\begin{cases} \text{Var}(A+B) = \text{Var}(A) + \text{Var}(B) \\ \text{Var}(A \cdot B) = \text{Var}(A) \cdot \text{Var}(B) + E[A]^2 \text{Var}(B) + E[B]^2 \text{Var}(A) \end{cases}$

(평균 0 $\rightarrow E[A]=0, E[B]=0 \Rightarrow \text{Var}(A) \cdot \text{Var}(B)$)

$\rightarrow Y = \sum_{i=1}^{n_{in}} w_i x_i$ 대입,

$$\text{Var}(Y) = \text{Var}(w_1 x_1 + w_2 x_2 + \dots + w_{n_{in}} x_{n_{in}})$$

$$\text{Var}(Y) = \text{Var}(w_1 x_1) + \text{Var}(w_2 x_2) + \dots + \text{Var}(w_{n_{in}} x_{n_{in}})$$

$$= \text{Var}(w_1) \text{Var}(x_1) + \dots + \text{Var}(w_{n_{in}}) \text{Var}(x_{n_{in}})$$

$$\Rightarrow \text{Var}(Y) = n_{in} \cdot \text{Var}(W) \cdot \text{Var}(X_{in})$$

세이버 $\text{Var}(W) = \frac{1}{n_{in}} \Rightarrow \text{Var}(Y) = \frac{1}{n_{in}} \text{Var}(X_{in})$

카이밍 $\text{Var}(W) = \frac{2}{n_{in}} \Rightarrow \text{Var}(Y) = \text{Var}(X_{in})$

앞층의 데이터 분산의 크기가 다음층에서도 그대로 보존

<Adam 옵티마이저>

Setting

W: 특정 가중치
t: 현재 에폭 (시간)
g_t: 현재 손실함수 미분값

보통 0.9로 설정
(β₁): 감소 계수 (누적된 방향 기록 90% 반영하고, 새로 계산된 미분방향 10% 반영)
→ 데이터 노이즈로 인해 미분값이 순간 변동한 곳으로 튜터라도, 관성 힘에 의해 모델이 안정적인 방향으로 유지하며 전진

보통 0.999
(β₂): 최근 미분 변동 폭의 제곱 99.9% 누적 반영 계수 (Adaptive Learning Rate)
→ 이 값이 최종 수식의 분모로 들어감으로써,
과거 미분값이 커서 격렬하게 움직였던 가중치는 보폭을 줄여주고,
미분값이 작아 거의 움직이지 못했던 가중치는 보폭을 늘려줌

① 1차 모멘텀 m_t 계산 (관성과 방향 제어)

→ 과거 이동하던 가중치의 속도와 방향 소실x 물리적 관성 유지

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

② 2차 모멘텀 v_t 계산 (가중치별 맞춤형 보폭 제어)

→ 기존 미분값들에 99.9%를 기저로 두고 새로 들어온 값에는 0.001의 미미한 무게만 부여

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

g_t²인 이유 → 2차 모멘텀에서는 방향과 관계없이 그 가중치가 겪은 '순수한 크기'를 측정하기 위해서

④ 그리고 어차피 최종식에서 $\sqrt{v_t}$ 하기 때문에 미분 크기는 변함없이 분모에 존재

③ 초기 편향 수정

→ 학습 초기 (t=1, t=2)에는 m₀, v₀가 0이기 때문에 계산값이 지나치게 0에 치우침 이를 위해

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

• t → ∞ 로 흐르면 어차피 분모가 1로 수렴

④ 최종 가중치 업데이트 공식

$$W_{t+1} = W_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

η: 기본 학습률

ε: 분모가 0이 되는 것을 방지하는 상수

< 최종 가중치 업데이트 수식 유도 >

$$m_t = (1 - \beta_1) g_t + \beta_1 m_{t-1}$$

$$\begin{aligned} \rightarrow m_t &= (1 - \beta_1) g_t + \beta_1 [(1 - \beta_1) g_{t-1} + \beta_1 m_{t-2}] \\ &= (1 - \beta_1) g_t + \beta_1 (1 - \beta_1) g_{t-1} + \beta_1^2 m_{t-2} \end{aligned}$$

$$\Rightarrow m_t = (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} g_i + \beta_1^t m_0$$

여기서 $m_0 = 0$ 이므로

$$\underline{m_t = (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} g_i}$$

통계적 기댓값 분석: 현재 m_t 값 $E[m_t]$ 가 진짜 이분값 $E[g_t]$ 와 일치하도록

$$E[m_t] = E \left[(1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} g_i \right]$$

$$\begin{aligned} \rightarrow E[m_t] &= (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} E[g_i] \\ &= E[g_t] \cdot (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} \end{aligned}$$

여기서 등비급수 합 공식을 이용해서

$$\sum_{i=1}^t \beta_1^{t-i} = \frac{1 - \beta_1^t}{1 - \beta_1} \text{로 변환 가능}$$

$$\rightarrow E[m_t] = E[g_t] \cdot \cancel{(1 - \beta_1)} \cdot \left(\frac{1 - \beta_1^t}{\cancel{1 - \beta_1}} \right)$$

$$\underline{E[m_t] = E[g_t] \cdot (1 - \beta_1^t)}$$

이를 통해 m_t 값이 진짜 이분값에 비해 $(1 - \beta_1^t)$ 만큼 차이가 난다는 것을 알 수 있다.

그래서
 $\Rightarrow$ 초기 편향 수식이 $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$

$$+ \hat{v}_t = \frac{v_t}{1 - \beta_1^t} \text{ 이다. } //$$

그리고 결국 $E[g_t] = \frac{E[m_t]}{1 - \beta_1^t}$ 이고

기존 경사하강법식은 $W_{t+1} = W_t - \eta \cdot g_t$ 이므로

기존 이분방향 $g_t \rightarrow \hat{m}_t$ 대입

그리고 $\hat{m}_t$ 로 인한 단위소거를 위해 $\rightarrow$ 분모에 $|\hat{v}_t|$ 를 나누어 줌 (gradient)

따라서

$$\Rightarrow \therefore W_{t+1} = W_t - \eta \frac{\hat{m}_t}{|\hat{v}_t| + \epsilon}$$

① ΔW 와 η 의 단위는 같음

(11/17)

< 전체적인 수학적·구조적 원리 및 과정 >

초기 설계 [차원 위주 행렬 계산]

① 입력층 → 첫 번째 은닉층

입력: $X \in \mathbb{R}^{200 \times 1}$

연산: $z^{(1)} = X \cdot W^{(1)} + b^{(1)}$

$W^{(1)}$ 차원: $[1, 64]$

$b^{(1)}$ 차원: $[64]$, 파이토치의 * 브로드캐스팅

* 브로드캐스팅: 파이토치나umpy가 부족한 차원을 가산으로 늘려 연산 크기를 맞추는 것

→ 왜?: 새로 만들어서 더하려면 메모리가 낭비됨

출력 $z^{(1)}$ 차원: $[200, 1] \times [1, 64] = [200, 64]$

활성화 함수: $A^{(1)} = \text{ReLU}(z^{(1)}) = \max(0, z^{(1)})$ (차원 변경 X)

② 2, 3, 4 hidden Layer

연산: $z^{(l)} = A^{(l-1)} \cdot W^{(l)} + b^{(l)}$ ($l=2, 3, 4$)

$W^{(l)}$ 차원: $[64, 64]$

$b^{(l)}$ 차원: $[64]$

출력 $A^{(l)}$ 차원: $[200, 64] \times [64, 64] = [200, 64]$

③ Output Layer

연산: $\hat{y} = z^{(5)} = A^{(4)} \cdot W^{(5)} + b^{(5)}$

$W^{(5)}$ 차원: $[64, 1]$

$b^{(5)}$ 차원: $[1]$

최종 출력 $\hat{y}$ 차원: $[200, 64] \times [64, 1] = [200, 1]$

< 순전파 및 손실함수 >

$$\hat{y}_{\text{pred}} = \text{model}(x)$$

$$\text{loss} = \text{criterion}(\hat{y}_{\text{pred}}, y)$$

실제해답은

각 레이어들을 거쳐

예측값($\hat{y}$)을 구하고

이것을 바탕으로 loss 함수 계산

$\text{nn.MSELoss}()$

$$\rightarrow L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

($N=20071$)



`optimizer.zero_grad()` & `loss.backward()`

< 역전파 과정과 체인룰 >

→ 출력층 가중치 $W^{(5)}$ 에 대한 미분

$$\frac{\partial L}{\partial W^{(5)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial W^{(5)}}$$

$$\frac{\partial L}{\partial \hat{y}} = \frac{2}{N} (\hat{y} - y)$$

$$\frac{\partial \hat{y}}{\partial W^{(5)}} = \frac{\partial}{\partial W^{(5)}} (A^{(4)} \cdot W^{(5)} + b^{(5)}) = A^{(4)}$$

$$\text{결과: } \frac{\partial L}{\partial W^{(5)}} = \underbrace{A^{(4)T}}_{[200, 1]} \cdot \underbrace{\left[\frac{2}{N} (\hat{y} - y) \right]}_{[64, 1]} = [64, 1] \Rightarrow$$

즉, 200개의 데이터의 전체가 레이어 내의 64개 노드 각각에 미치는 오차 기여도

④ 내부 은닉층 가중치 $W^{(4)}$ 에 대한 미분 (ReLU 포함)

$$\frac{\partial L}{\partial W^{(4)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial A^{(4)}} \cdot \frac{\partial A^{(4)}}{\partial z^{(4)}} \cdot \frac{\partial z^{(4)}}{\partial W^{(4)}}$$

$$A^{(4)} = \max(0, z^{(4)}) \Rightarrow \frac{\partial A^{(4)}}{\partial z^{(4)}} = \begin{cases} 1 & (z^{(4)} > 0) \\ 0 & (z^{(4)} \leq 0) \end{cases}$$

이러한 과정을 통해

$W^{(1)} \sim W^{(5)}$ 의 미분값을

구할 수 있음

< 전체적인 수학적 구조적 원리 및 과정 >

< Adam Optimizer (Parameter Update) >

① 1차 모멘트 계산 (방향 및 관성 설정)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

② 2차 모멘트 계산 (가중치별 보폭 설정)

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

③ 최종식 :
$$W_{t+1} = W_t - \frac{\eta}{\sqrt{v_t} + \epsilon} \hat{m}_t$$



이후 Loss 오차 값을 최소로 줄여

정답률과 가까워지도록 AI를 학습시킨다.

효과: 기울기 변동이 심한 가중치는 분모($\sqrt{v_t}$)가 커져서 조금만 이동,

기울기 변동이 적은 가중치는 분모가 작아져 크게 이동,

여러한 과정을 통해 Local minima에 빠지지 않고

적절한 가중치를 구할 수 있다.

